

# SOCIAL MEDIA AND WEB ANALYTICS

Prof Dr Matthias Bogaert

# LECTURE 4: SENTIMENT ANALYSIS WITH TRADITIONAL APPROACHES

# CONTENT

- Lecture 5: Sentiment analysis with traditional approaches
  - Word-of-mouth analysis
    - WOM types
  - Sentiment analysis
    - Introduction
    - Lexicon-based approach
    - Machine learning approach

# WORD-OF-MOUTH ANALYSIS

# WOM TYPES

- Earned social media on social media is mostly called word-of-mouth (WOM)
- 4 main dimensions (the 4 Vs)
  1. Volume
  2. Valence
  3. Variance
  4. Variety

# WOM TYPES

## 1. Volume

- The total number of posts/tweets/comments/replies
- Influences performance (e.g., sales) via
  - Awareness effect
  - Herding effect
  - Bandwagon effect

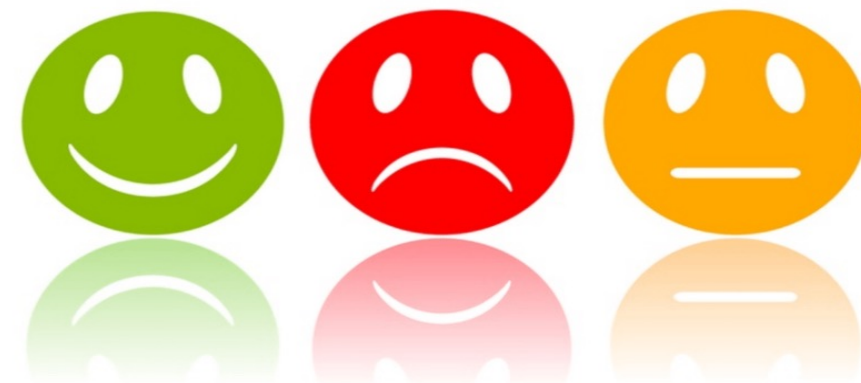
Positive  
relationship  
between the  
total number  
tweets and  
sales



# WOM TYPES

## 2. Valence

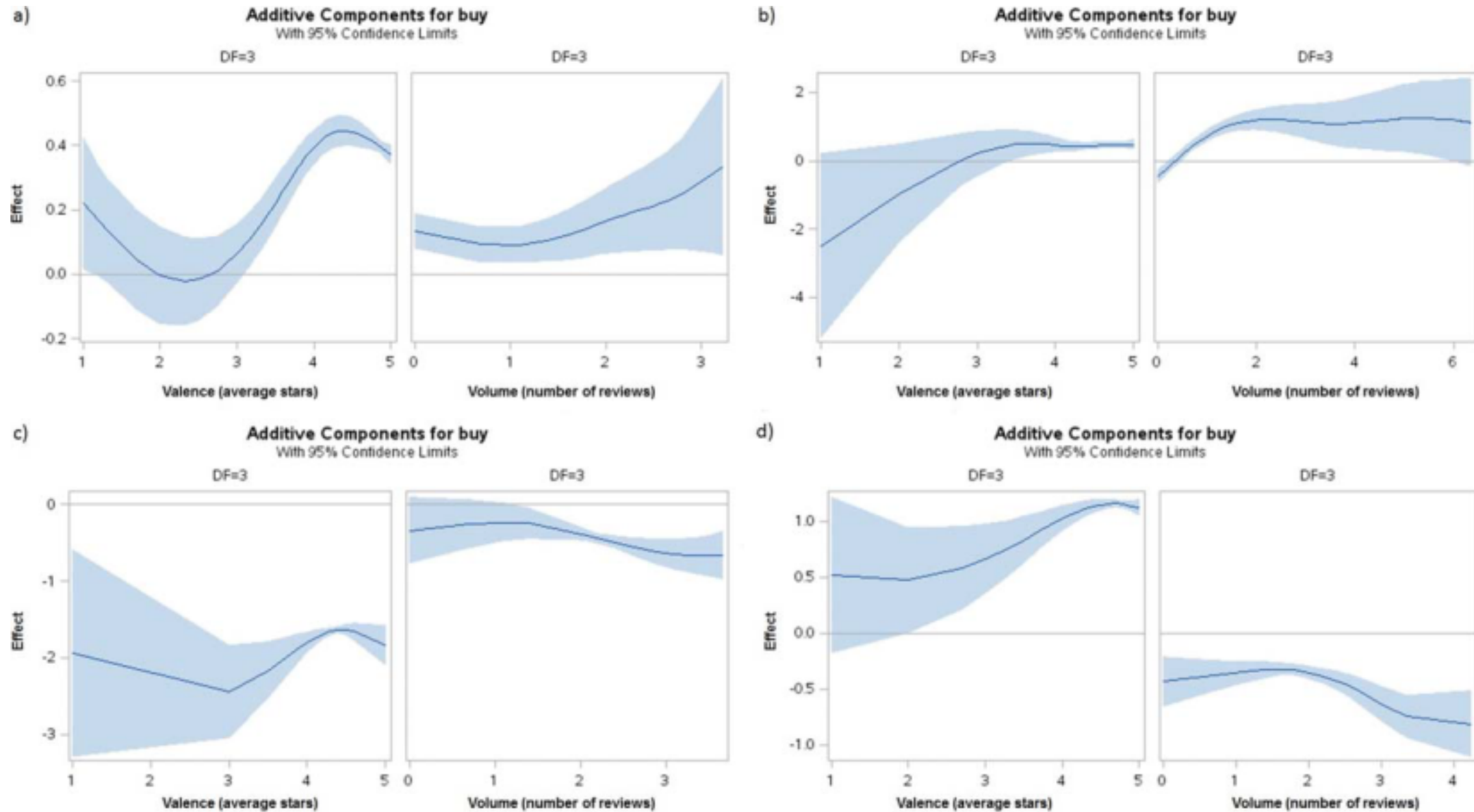
- Is WOM positive, negative, neutral, or mixed?
- How?
  - Consumer ratings
    - Too-good-to-be-true effect
  - **Sentiment analysis (cf infra)**
- Can be calculated in combination with volume
  - E.g., number of positive comments
- Influences sales via
  - Persuasiveness effect
- Different opinions about whether volume or valence is most important





# WOM TYPES

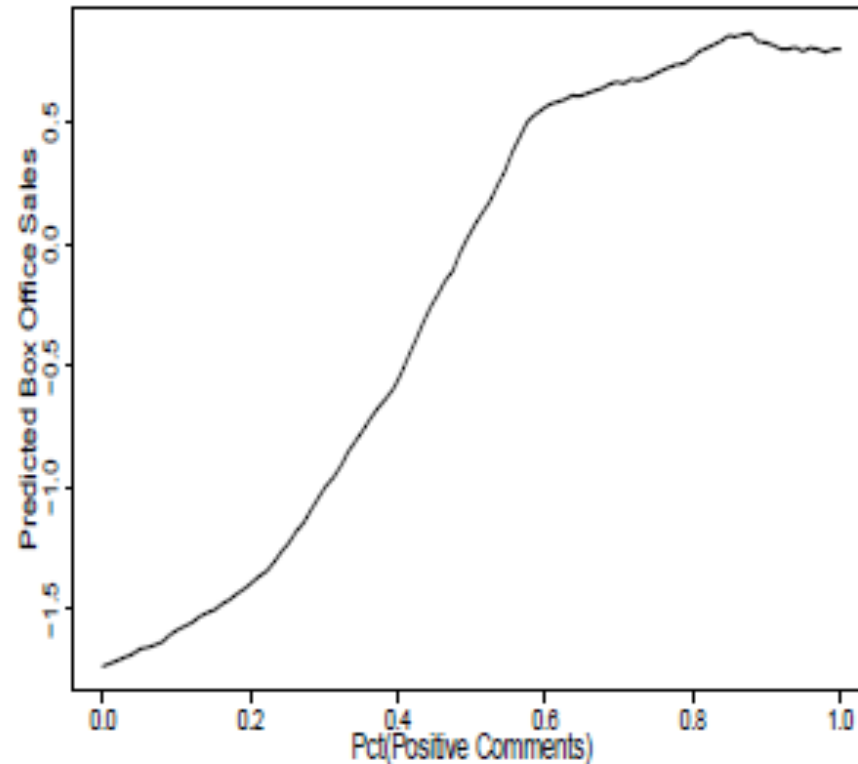
## 2. Valence



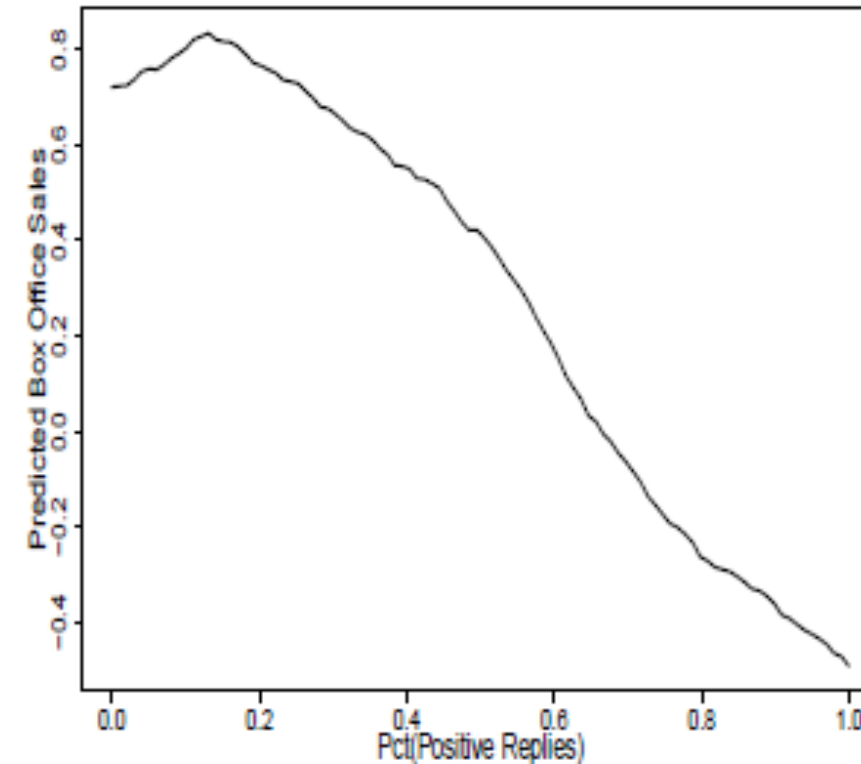


# WOM TYPES

## 2. Valence



(a) Relationship between box office sales and percentage of positive comments on Facebook before release



(b) Relationship between box office sales and percentage of positive replies on Twitter before release

# WOM TYPES

## 3. Variance

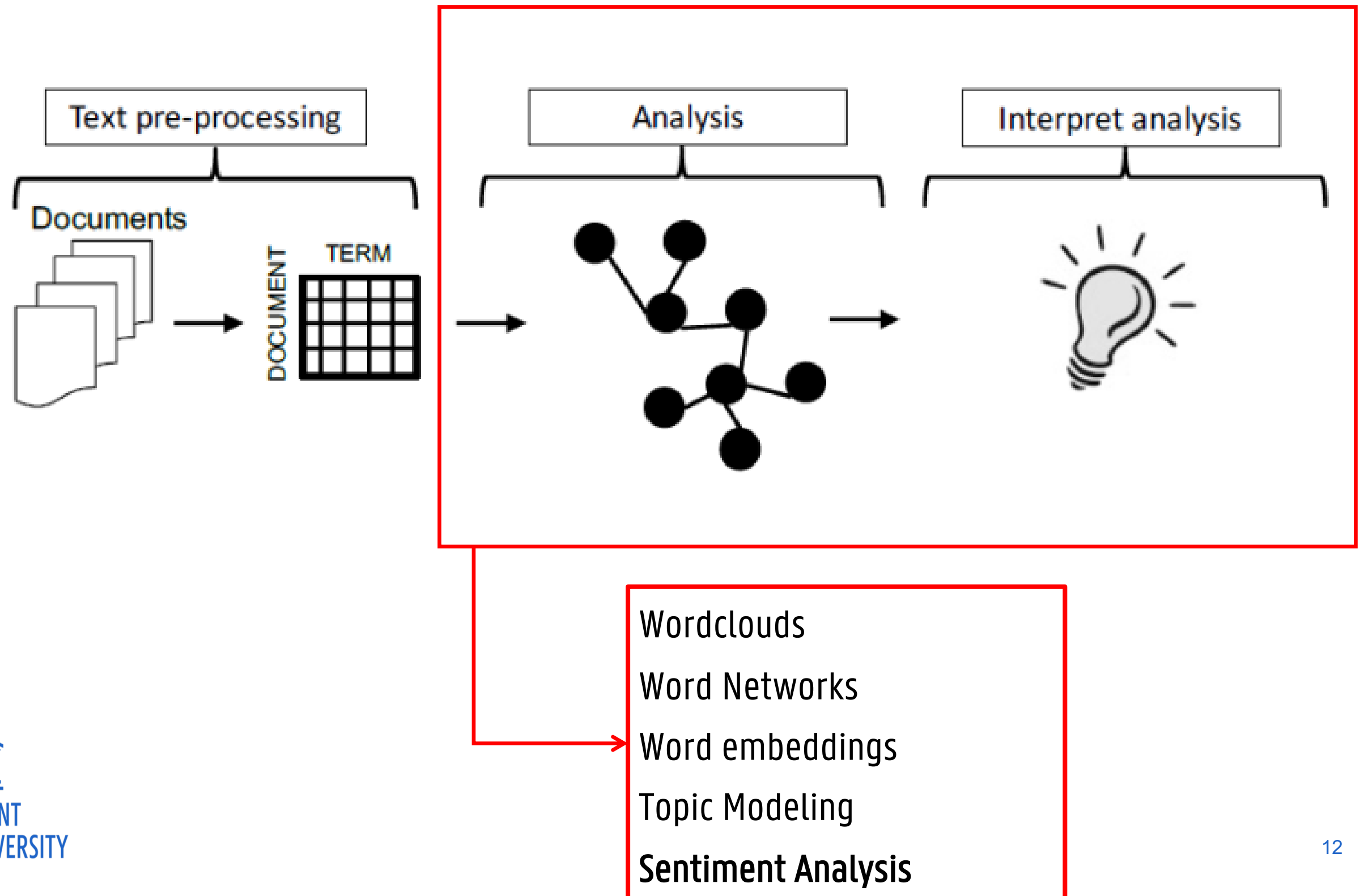
- Spread of communication across social groups/platform/time/...
- Measured as the variance of the valence across the posts/tweets/...

## 4. Variety

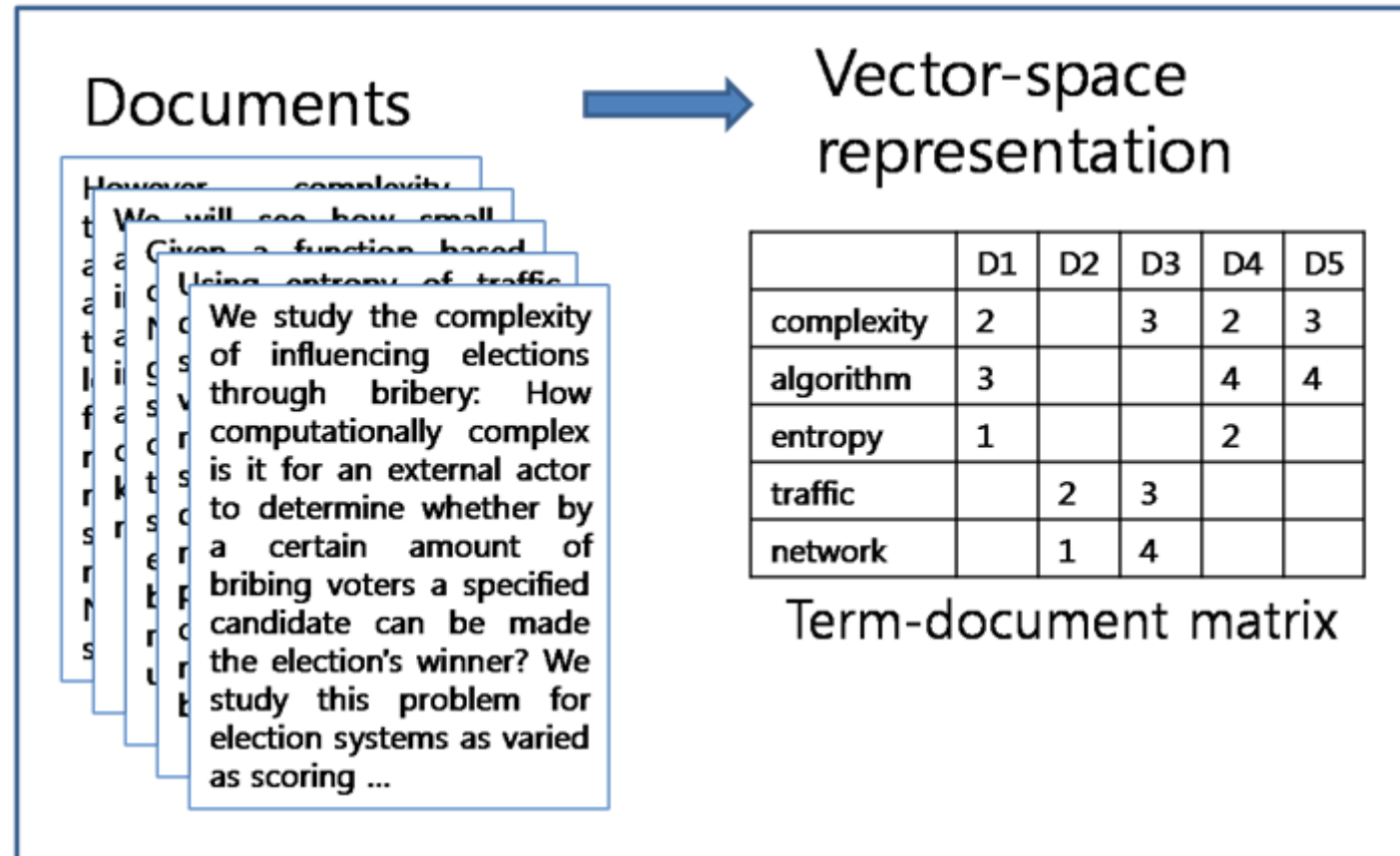
- Also called media richness
- Use of textual and visual content
- Richer and more personal media are more effective
- For example
  - Photos receive more likes and engagement than plain text, videos or links

# SENTIMENT ANALYSIS: INTRO

# SENTIMENT ANALYSIS: OVERVIEW

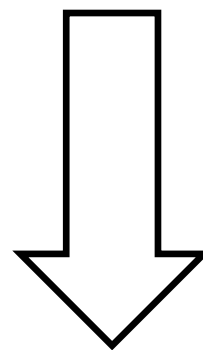


# TEXT MINING REFRESHER: DTM



# TEXT MINING REFRESHER: DTM

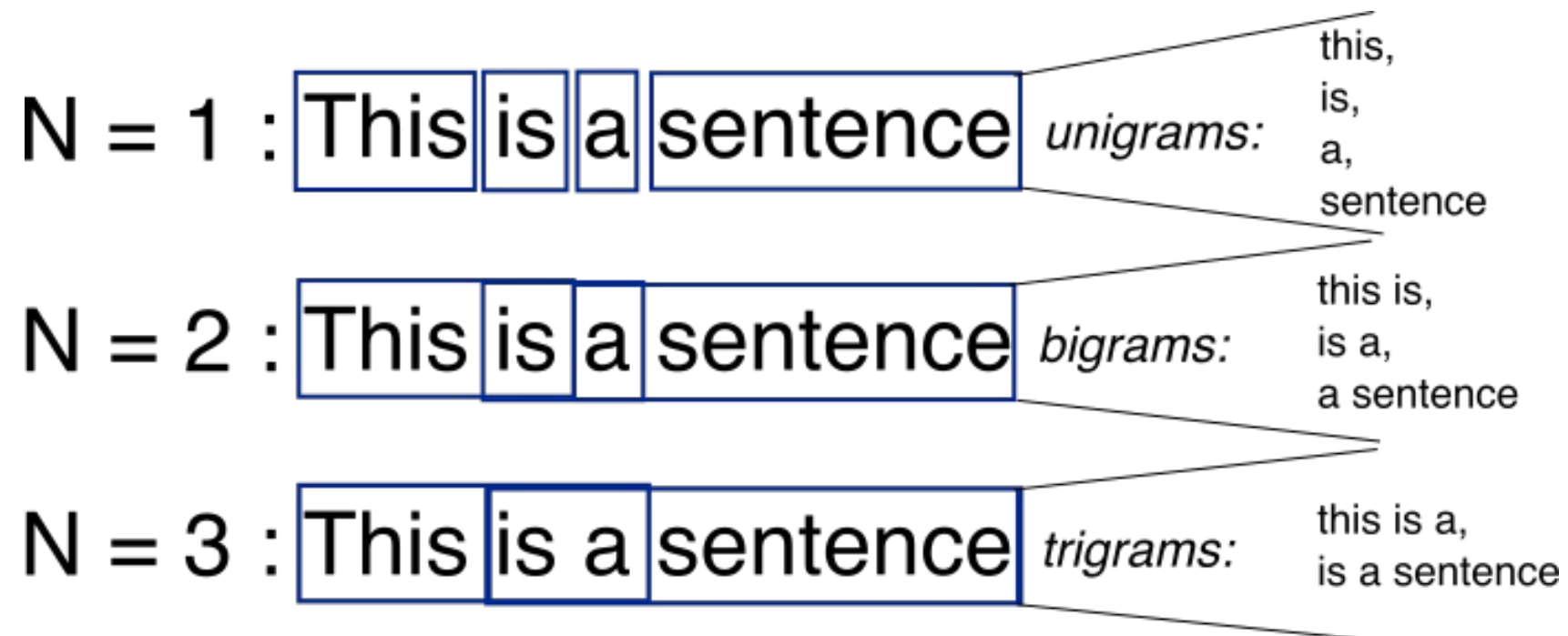
| ID | Suggestion                                 |
|----|--------------------------------------------|
| 1  | online banking could be more user-friendly |
| 2  | online banking services could be faster    |
| 3  | faster alerts, faster services             |



| ID | online | banking | could | be | more | user-<br>friendly | services | faster | alerts |
|----|--------|---------|-------|----|------|-------------------|----------|--------|--------|
| 1  | 1      | 1       | 1     | 1  | 1    | 1                 | 0        | 0      | 0      |
| 2  | 1      | 1       | 1     | 1  | 0    | 0                 | 1        | 1      | 0      |
| 3  | 0      | 0       | 0     | 0  | 0    | 0                 | 1        | 2      | 1      |

# N-GRAMS

- So far we have only focussed on unigram features (bag-of-words)
- However, bi-gram, tri-gram, ..., N-gram lexical features also exist





# N-GRAMS

- N-grams can strengthen your analysis
  - Negation
  - Intensifiers
  - Specific combinations of words
- No agreement in literature that N-gram features increase classification accuracy
- In sentiment analysis, n-grams are often needed to extract the correct sentiment ('valence shifters')



# SENTIMENT ANALYSIS: WHAT

- Sentiment analysis is the computational process of extracting sentiment from text
  - It tries to extract the polarity of an opinion ranging from positive to negative (~also called opinion mining)
  - It can be expressed as a score ranging from e.g. 1 to 9 (Likert scale) or -4 to 4
  - The sentiment of a text is also referred to as valence
- Located at the interface of natural language processing, computational linguistics and text mining
- Why is it important?
  - Companies are confronted with huge amounts of user-generated content (e.g., blog posts, online fora, social media, etc.)
  - Sentiment analysis enables companies to understand this unstructured data and gain insights into consumer behavior
  - Moreover, the process can be fully automated
- Applications
  - Finance, marketing, reviews, etc.

# SENTIMENT ANALYSIS: APPROACHES

## **1. Lexicon-based approach**

- Data: No labeled data needed
- Speed: Very fast execution, low computational cost
- Limit: Ignores context, fails with content nuances

## **2. Supervised learning approach (traditional and deep learning)**

- Data: Requires a labeled data set or manual labeling (high effort)
- Speed:
  - Traditional (e.g., SVM): Fast on CPU, but ignores word order (Bag-of-Words).
  - Deep learning (e.g., CNN BERT): Context-aware, but requires GPUs/heavy compute.
- Limit: Better context awareness but depends on data

## **3. Generative AI or LLM approach**

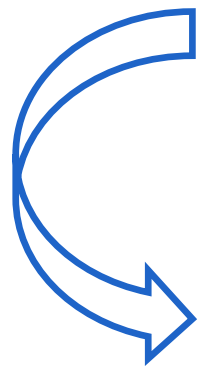
- Data: little to no labeled data needed (zero-shot or few-shot learning)
- Speed: Fast to implement, but computational or monetary cost
- Limit: understands nuances, but prompt-dependent and sensitive to hallucinations.

# LEXICON-BASED APPROACH

# LEXICON APPROACH

- Look up corresponding words in a sentiment dictionary
- Available in several languages
- Often expressed on a 7 or 9-point Likert scale
  - Final sentiment score is often rescaled (-4: not happy, 0: neutral, 4: happy)

|   | Word        | # VALENCE | # AROUSAL | # DOMINANCE |
|---|-------------|-----------|-----------|-------------|
| 0 | aardvark    | 6.26      | 2.41      | 4.27        |
| 1 | abalone     | 5.3       | 2.65      | 4.95        |
| 2 | abandon     | 2.84      | 3.73      | 3.32        |
| 3 | abandonment | 2.63      | 4.95      | 2.64        |
| 4 | abbey       | 5.85      | 2.2       | 5.0         |
| 5 | abdomen     | 5.43      | 3.68      | 5.15        |
| 6 | abdominal   | 4.48      | 3.5       | 5.32        |
| 7 | abduct      | 2.42      | 5.9       | 2.75        |
| 8 | abduction   | 2.05      | 5.33      | 3.02        |
| 9 | abide       | 5.52      | 3.26      | 5.33        |



|   | Word        | # VALENCE          | # AROUSAL           | # DOMINANCE         |
|---|-------------|--------------------|---------------------|---------------------|
| 0 | aardvark    | 1.259999999999998  | -2.59               | -0.7300000000000004 |
| 1 | abalone     | 0.299999999999998  | -2.35               | -0.0499999999999982 |
| 2 | abandon     | -2.16              | -1.27               | -1.6800000000000002 |
| 3 | abandonment | -2.37              | -0.0499999999999982 | -2.36               |
| 4 | abbey       | 0.849999999999996  | -2.8                | 0.0                 |
| 5 | abdomen     | 0.429999999999997  | -1.319999999999998  | 0.15000000000000036 |
| 6 | abdominal   | -0.519999999999996 | -1.5                | 0.32000000000000003 |
| 7 | abduct      | -2.58              | 0.9000000000000004  | -2.25               |
| 8 | abduction   | -2.95              | 0.33000000000000007 | -1.98               |
| 9 | abide       | 0.519999999999996  | -1.7400000000000002 | 0.33000000000000007 |

# LEXICON APPROACH

- Strategy
  - Look for a word in a sentence in the dictionary
  - Use the value of that word in the dictionary as sentiment value
  - Aggregate sentiment for the entire sentence (or for instance aggregate per day)
    - Mean of the sentiment of all words
    - Number (percentage) of positive/negative words

# LEXICON APPROACH

## — Example

- ‘I think the first course of analytics was good’
- Look for the words in the dictionary and assign sentiment score
  - ‘think’ = 1.68, ‘first’ = 2.33, ‘good’ = 2.89
  - Sentiment score of sentence 1 =  $(1.68 + 2.33 + 2.89)/3 = 2.3$
- If there are no matches found, assign a value of 0 (neutral)

# LEXICON APPROACH: PROBLEMS

- Pitfalls when using social media data
  - What to do with dirty text?
  - What to do with emoticons?
  - How to add negation?
  - How to add intensifiers?





# LEXICON APPROACH: TEXT CLEANING

- Careful preprocessing of the text is necessary!
- Normalization
  - Remove special characters, links, stop words,...
  - However, punctuation, upper cases and emoticons can be meaningful
- Replace the emoticons with their corresponding meaning
  - For emojis, replace it with their unicode (<https://raw.githubusercontent.com/today-is-a-good-day/emojis/master/emojis.csv>)
  - Wikipedia: [https://en.wikipedia.org/w/index.php?title=List\\_of\\_emoticons&oldid=900482232](https://en.wikipedia.org/w/index.php?title=List_of_emoticons&oldid=900482232)

| Icon |     |     |     |     |     |     |     |     |    |    | Emoji           | Meaning                                                                                                                      |
|------|-----|-----|-----|-----|-----|-----|-----|-----|----|----|-----------------|------------------------------------------------------------------------------------------------------------------------------|
| :~)  | :~] | :~3 | :~> | 8~) | :~} | :o) | :c) | :^) | =] | =) | 😊 😄 😁 😊 😇       | Smiley or happy face. <sup>[4][5][6]</sup>                                                                                   |
| :~D  | 8~D | x~D | X~D | =~D | =~3 | B^D |     |     |    |    | 😂 😄 😁 😊 😇       | Laughing, <sup>[4]</sup> big grin, <sup>[5][6]</sup> laugh with glasses, <sup>[7]</sup> or wide-eyed surprise <sup>[8]</sup> |
| :~)) |     |     |     |     |     |     |     |     |    |    |                 | Very happy or double chin <sup>[7]</sup>                                                                                     |
| :~(  | :~c | :~< | :~[ | :~  | >~[ | :~{ | :~@ | >~( |    |    | 😞 😟 😠 😡 😢 😣 😤 😥 | Frown, <sup>[4][5][6]</sup> sad, <sup>[9]</sup> angry, <sup>[7]</sup> pouting                                                |
| :~-( |     |     |     |     |     |     |     |     |    |    | 😭 😢             | Crying <sup>[9]</sup>                                                                                                        |



# LEXICON APPROACH: TEXT CLEANING

- Replace slang, abbreviations, word elongations, kerning
- Spell check the words
- Apply lemmatization
  - Stemming would not be advised since we want a match with our lexicon

# LEXICON APPROACH: NEGATION

- Split up text in only unigrams, bigrams, and a combination
  - Example: 'This is not good'
  - Unigram: 'this', 'is', 'not', 'good'
  - Bigram: 'this is', 'is not', 'not good'
  - Combination: 'this', 'is', 'not', 'good', 'this is', 'is not', 'not good'
- If there is a negation, the sentiment score should be flipped
  - For example, 'happy' has a score 7 on a 9-point Likert scale → 'not happy' should receive a score of 3 ( $=10-7$ )
- However, if there is a match for a negation, there will also be a match for the original word
  - For example, if 'not good' is matched, then 'good' will also be match in the lexicon.
  - Hence, make sure that the combination of 'good' and 'not good' equals 3.

# VADER

- **VADER** (Valence Aware Dictionary and sEntiment Reasoner)
  - Probably the most popular sentiment analysis package in **Python**
  - **vaderSentiment** (<https://github.com/cjhutto/vaderSentiment>)
  - A lexicon and rule-based sentiment analysis tool specifically attuned to social media text (but applicable in other domains as well)
  - Considered the **gold standard lexicon**
    - Built on a massive dictionary of over 9,000 tokens.
    - Human-labeled: Every token was rated by 10 independent human raters
    - Raters scored words on a scale from -4 (Extremely Negative) to +4 (Extremely Positive)
  - Other characteristics
    - Computationally efficient: Extremely fast execution compared to Deep Learning models
    - Ready-to-use (zero-shot): No training data required, it works right out of the box
    - Context-aware: Takes into account negation, intensifiers, understands emoticons/emojis, acronyms (LOL, ROFL), and slang (meh, nah)

# VADER

- The strength of VADER lies in the use of **valence shifters**
  - A valence shifter changes the sentiment, either by intensifying, dampening or flipping the polarity
  - To implement valence shifters, it looks for the tri-gram (3 words) preceding a sentiment word
- **Amplifier: Increases intensity**
  - Words
    - *"The service was extremely good"*
  - Punctuation
    - Exclamation marks (!) increase the magnitude of the intensity
    - *"The food is hot" vs. "The food is hot\*\*!!!!\*\*"*
  - Capitalization
    - All-caps increases the sentiment score in the presence of non-capitalized words
    - *"The food is great" vs. "The food is GREAT"*
- **De-amplifier: Decreases intensity**
  - *"The service was kind of good"*
- **Adversative conjunctions**
  - The word "but" or "however" signals a shift in sentiment. VADER lowers the weight of the text before "but" and increases the weight after it
  - *"The plot was boring, but the acting was phenomenal!"*
- **Negator**
  - Flips the sentiment of the word
  - *"The food is not good"*

# VADER

- VADER outputs the following things: `{ 'neg' : 0.0, 'neu' : 0.3, 'pos' : 0.7, 'compound' : 0.85 }`
- **pos, neu, neg**
  - These represent the proportion of the text that falls into each category and they sum up to 1
  - Useful for analyzing the structure of the text, but not the overall sentiment
- **compound**
  - A single, unidimensional measure of sentiment
  - Calculate the sum of valence scores ( $x$ ) for all words, including adjustments valence shifters
  - Apply the following **normalization formula** to map the sum between -1 and +1:
$$\text{NormalizedScore} = \frac{x}{\sqrt{x^2 + \alpha}}$$
  - $\alpha$ : Default is 15. This heuristic value approximates the maximum expected valence sum for a typical sentence
  - **Thresholds:** Positive  $\geq 0.05$ , Neutral between -0.05 and 0.05, Negative  $\leq -0.05$

- **TextBlob** is portmanteau Python library (built on top of NLTK and Pattern) for processing textual data
- It is designed for a simple and consistent API to perform common NLP tasks
- Key capabilities:
  - **Sentiment analysis:** polarity and subjectivity scoring
  - **Noun phrase extraction:** Identifying the noun phrases in a sentences (= the key subjects)
  - **POS tagging:** Labeling nouns, verbs, etc.
  - **Lemmatization:** Transform inflected word forms to its root
  - **Tokenization:** Splitting the text with various word tokenizers
  - **Spelling correction:** Automated spelling correction and pluralization

# TEXTBLOB

- TextBlob's default analyzer uses a rule-based lexicon (from the Pattern library) to calculate two specific scores:
  - **Polarity** [-1.0, 1.0]: Measures emotional orientation (+1 is positive, -1 is negative).
  - **Subjectivity** [0.0, 1.0]: Measures fact vs. opinion (0 is objective, 1 is highly subjective).
- **Sentiment calculation**
  - The lexicon contains ~3,000 words pre-tagged with scores. It calculates the arithmetic mean of all sentiment words in your text.
  - **Intensifiers**: Adverbs like *"very"* act as multipliers (multiply by 1.3), boosting the polarity of the following adjective
  - **Negations**: When it detects words like *"not"* or *"never"*, it multiplies the next word's polarity by -0.5 (flipping the sentiment).



# TEXTBLOB

- Besides the default rule-based analyzer, TextBlob also allows you to swap the engine for a **supervised learning model**
- A naive Bayes classifier trained on the NLTK "movie\_reviews" corpus.
  - Instead of summing up word scores, it calculates the probability that a document belongs to a category based on word frequencies
- It treats sentiment as a **classification problem** rather than a continuous scale.
  - **Classification**: Returns a discrete label (pos or neg)
  - **Probability**: Returns the confidence level for the prediction (e.g., p\_pos: 0.95, p\_neg: 0.05)
- Notes
  - **Domain-specific**: Since it is trained specifically on movie reviews, it may perform poorly on tweets, news, or general text
  - **Performance**: It is significantly less performant and slower than the default rule-based analyzer



# TEXTBLOB VS VADER

| Feature          | TextBlob (Default)                                                        | VADER                                                                               |
|------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Primary focus    | General purpose text handling                                             | Sentiment with focus on social media (tweets, comments, chats)                      |
| Valence shifters | Handles (de-)amplifiers, negation, punctuation, capitalization, emoticons | More advanced handling (tri-grams)<br>Emojis 😊 are handled naturally                |
| Output           | Polarity & subjectivity                                                   | Pos/Neu/Neg scores & compound score                                                 |
| Vocabulary       | Basic English<br>Unkown words are ignored                                 | Tuned to social media with slang and acronyms included<br>Unknown words are ignored |
| Performance      | Fast                                                                      | Very fast                                                                           |

# NRCLEX: DETECTING EMOTIONS

- While TextBlob/VADER measure sentiment polarity, **NRClex** maps text to the Plutchik Wheel of Emotions
- NRClex detects 8 basic **emotions**:
  - **Anger**: Rage, Fury, Outrage
  - **Fear**: Terror, Anxiety, Panic
  - **Anticipation**: Expectancy, Interest, Vigilance
  - **Trust**: Acceptance, Admiration, Faith
  - **Surprise**: Amazement, Astonishment, Shock
  - **Sadness**: Grief, Sorrow, Gloom
  - **Joy**: Ecstasy, Elation, Delight
  - **Disgust**: Loathing, Revulsion, Aversion
- You can also derive a **sentiment score**: positive word count - negative word count
- The original data source is based on the **NRC Emotion Lexicon** (aka **EmoLex**): a list of ~14,000 words manually tagged with emotions via crowdsourcing (Mohammad & Turney, 2013)
- The **NRClex** library expands this list to ~27,000 words by finding synonyms using NLTK's WordNet. This allows for detection of emotions even if the exact word isn't in the original lexicon.

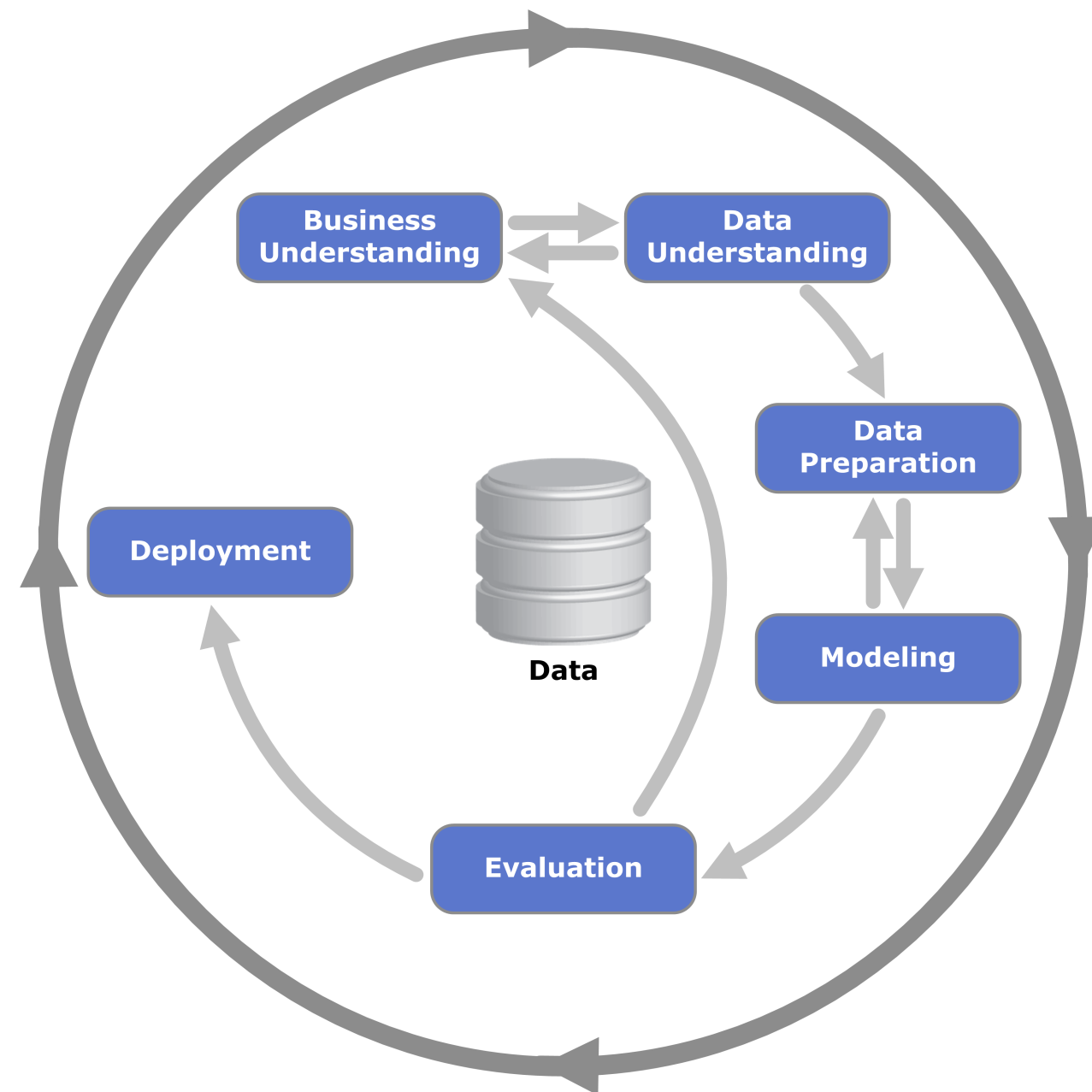
# TRADITIONAL MACHINE LEARNING

# APPROACH

- Use various text features as an input to train a machine learning model
- The machine learning model is then used to ‘predict’ sentiment
- They need the sentiment labels (response variable)
  - Difficult to acquire and time- or money-consuming
- In general, they are more accurate, but also they do not scale well
- They also do not generalize and have to be designed for specific application
  - They are black-boxes
- Follow all the steps of the CRISP-DM model (Cross-industry Standard Process for Data Mining)

# APPROACH

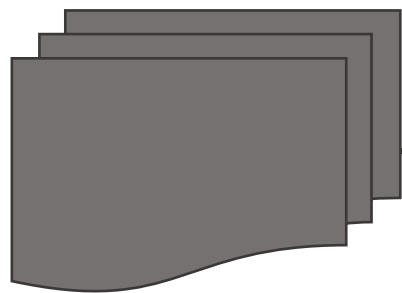
- CRISP-DM model (Cross-industry Standard Process for Data Mining)



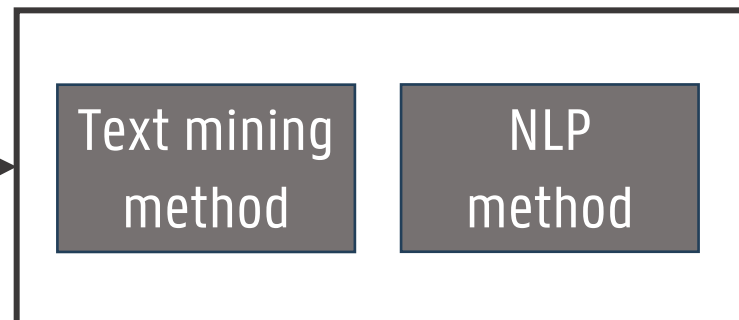
# APPROACH

Downstream task

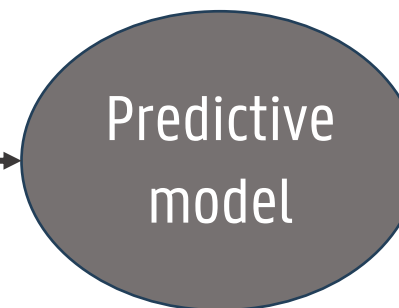
Social media data



Feature engineering



Modeling



Sentiment analysis

Or any other textual data

- Product reviews
- Repair notes
- Emails
- Websites
- Financial reports/statements
- ...

- LR
- RF
- XGB
- NN
- ...

Or any other task

- helpfulness
- Maintenance
- Churn
- New leads
- Fraud
- ...

# PREPROCESSING

## — Data preparation

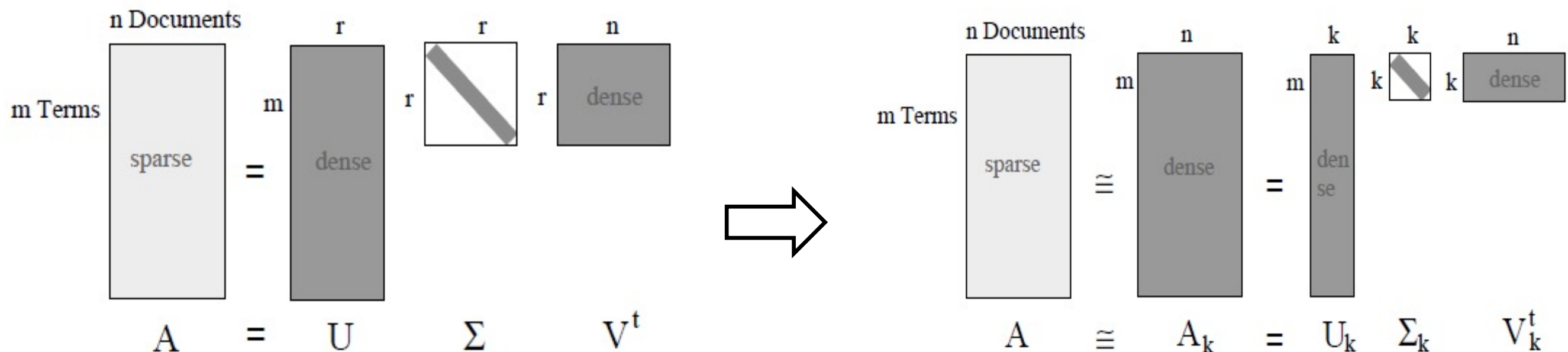
- Type of features (independent variables):
  - Lexicon features
  - Lexical features
    - E.g., Bag of words, n-grams, concepts, topics, word2vec, etc.
  - Syntactic (or linguistic) features
    - E.g., Part-of-Speech, number of words, exclamation marks, ...
  - Other information related to the text
    - E.g., time-related information of a Tweet
- Literature has focused most on lexical features



# PREPROCESSING

## — Data preparation

- All the preprocessing steps to come up with the document-by-term matrix
- Reduce the dtm by using latent semantic indexing (LSI)
  - Apply singular vector decomposition (SVD) to the DTM
  - Select the first  $k$  principal component directions





# PREPROCESSING

## — Data preparation

- To predict sentiment, you need a set of labelled documents
  - Binary: positive/negative
  - Multi-class: positive/neutral/negative
- These documents need to be manually labelled
- Only a subset of the documents can be labelled



# PREPROCESSING

## — Data preparation

- Raters should read the text and evaluate its sentiment
- Can be different across raters → Calculate interrater agreement (Fleiss Kappa)

| K           | Interpretation           |
|-------------|--------------------------|
| $< 0$       | Poor agreement           |
| 0.01 – 0.20 | Slight agreement         |
| 0.21 – 0.40 | Fair agreement           |
| 0.41 – 0.60 | Moderate agreement       |
| 0.61 – 0.80 | Substantial agreement    |
| 0.81 – 1.00 | Almost perfect agreement |

# PREPROCESSING

## — Data preparation

- Another way to label the dependent variable is to use emoticons in the text to label the text as positive negative
  - Often applied in social media
  - Only possible for text with emoticons
  - For example, 'I really like the course on text mining :D :D, but it is really difficult ☹'
  - → Two positive emoticons, one negative → text is labeled as positive
- You still need to manually check whether there is interrater agreement between the labels



# SET-UP

## — Data preparation

- Only a subset of the data is labeled
- Labeled data need to be split into training and test set
- Make sure the test and training set contain the same terms
  - For the dtm
    - Use the same terms in the test as in the training set
  - For the SVD
    - Use the concept loadings from the training set
  - For word embeddings
    - Use the same embedding structure in the training and test set

# SET-UP WITH SVD

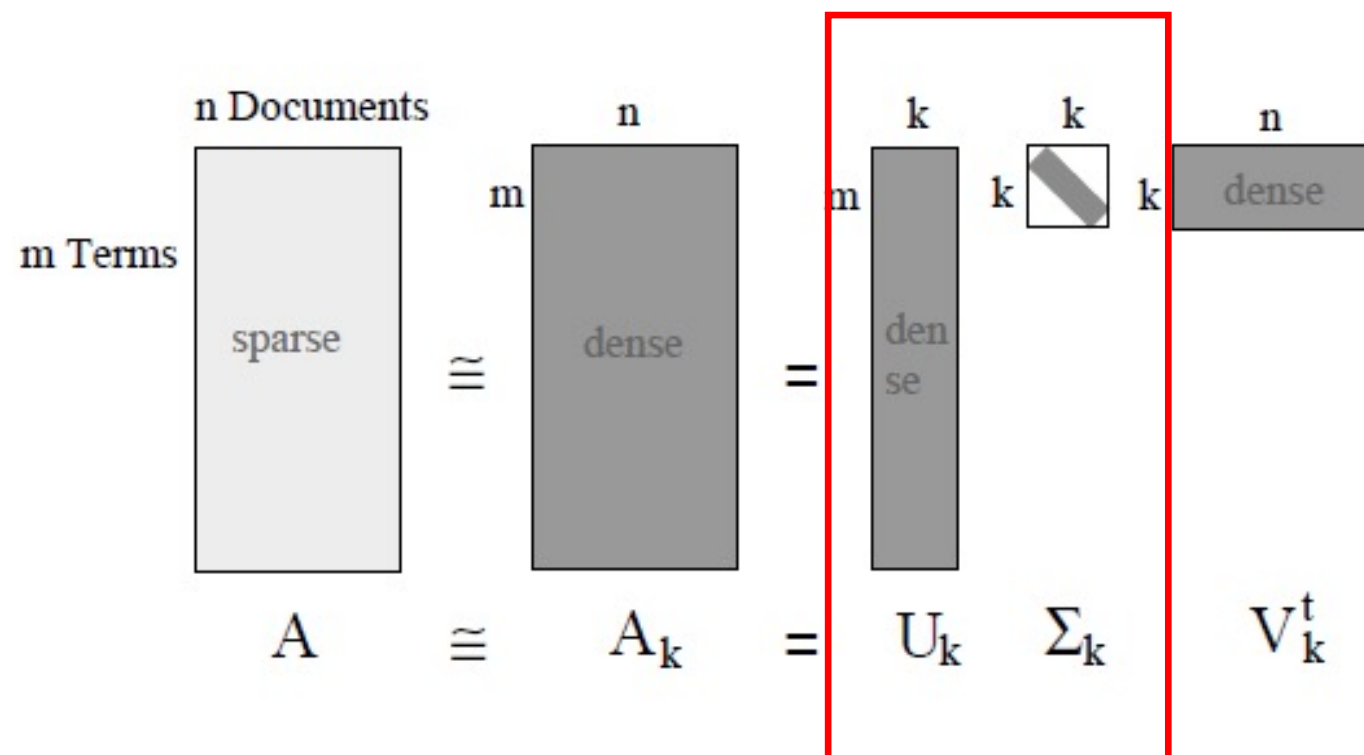
## — Data preparation

— For the SVD: use the concept loadings from the training set

— Use the formula

—  $V_{val} = A_{val}^t U_k \Sigma_k^{-1}$

— With  $V_{val}$  the document-by-concept matrix for the validation set,  $A_{val}^t$  the transposed validation term-by-document matrix,  $U_k$  the  $k$ -reduced term-by-concept matrix and  $\Sigma_k^{-1}$  the inverse of the  $k$ -approximation of  $\Sigma$ .



# MACHINE LEARNING MODELING

## — Modeling

- Dependent variable: sentiment
  - Can be binary (positive or negative) or multi-class (positive, neutral, negative)
- Independent variables
  - Outcome of text mining
  - Other variables can also be added
  - Lexicon-based sentiment can be added as well
- Algorithms
  - Traditional machine learning models
  - Deep learning
  - Transformers

# MACHINE LEARNING MODELING

## — Traditional machine learning models

- Input: raw text needs to be transformed to a correct representation on document level or your level of analysis
  - Bag of words
  - TF-IDF (with or without matrix factorization)
  - Embeddings
    - Often Doc2vec (~generalization of word2vec to documents)
- Models
  - Naive Bayes: fast and easy-to-use algorithm with moderate performance
  - Logistic regression
  - Support vector machines: works well with high-dimensional data
  - Random forest
  - XGBoost and friends (lightGBM, catboost, etc.)
  - Artificial neural network
- Performance is competitive, although not as good as state-of-the-art transformers
- Experimentation with different text representations is crucial!



# WORD EMBEDDINGS

## — Working with embeddings

- Word embeddings should be considered as 'lexical' features just like BoW, TF-IDF, SVD concepts or LDA topics
- Once they are created, embeddings can be inputted in any machine learning model
- For traditional machine learning models
  - Pre-trained embeddings are needed!
  - Word embeddings are often aggregated on document level = Doc2Vec or Paragraph Vector - Distributed Memory (PV-DM) model



# FROM WORD TO DOCUMENT EMBEDDINGS

- To analyze on a document-level, we must combine the pre-trained word embeddings into a single document embedding ( $d$ )
- There are several strategies to do this:
  - **Simple average**
    - Take the average of all word vectors in the document
    - A 5-word review and a 500-word review would look mathematically the same
  - **Sum + L2 normalization**
    - Sum the vectors, then shrink or stretch them so they all have a **geometric length** of exactly 1
    - Again, it forces a short review to have the same mathematical "weight" as a detailed review, losing valuable context
  - **Weighted average (e.g., TF-IDF)**
    - Give unique/rare words more weight, then average them
    - This helps highlight keywords, but it still ignores the total amount of information in the text

# FROM WORD TO DOCUMENT EMBEDDINGS

## — Our approach: Vector space scale normalization

- Document vectors are the sum of the vectors of the words in the document, standardized by the scale of the vector space.
- Document Vector ( $d$ ):
  - $d = \frac{\sum_{i \in doc} w_i}{S}$
- Vector space scale ( $S$ ): Defined as the square root of the average inner product of all vectors in the vocabulary
  - $S = \sqrt{\frac{1}{|V|} \sum_{j \in V} (w_j \cdot w_j)}$
- Why this approach?
  - **Captures magnitude:** Summing preserves the total information content (unlike averaging)
  - **Geometry aware:** Normalizing by the vector space scale ensures the vector is scaled relative to the intensity of the pre-trained model, avoiding arbitrary scaling issues

# FASTTEXT: ALTERNATIVE WORD EMBEDDINGS

- While Word2Vec learns vectors for whole words, **FastText** (Bojanowski et al., 2013) breaks words down into smaller chunks called character n-grams.
  - It adds .< and .> to mark the start and end
  - It breaks every word down into sliding windows of characters (n-grams) and the whole word itself.
  - The final vector is the sum of all these components of the sliding window approach



# FASTTEXT: ALTERNATIVE WORD EMBEDDINGS

- Because FastText learns sub-words, it can construct meaning for words it has never seen before (**out-of-vocabulary words**)
- Handling of typos
  - Input: "guggle" (instead of "google")
  - Word2Vec: Unknown
  - FastText: It recognizes the n-grams gug, ugg, ggl, gle from other words like "struggle" or "goggles" and constructs a meaningful vector
- Understanding compound words
  - Inputs: "tensorflow"
  - Word2Vec: Unknown (for a model trained in 2013)
  - FastText: It recognizes tensor (math object) + flow (movement). Even if the specific brand name is missing, the semantic meaning is preserved.
- Since social media data is often noisy text, we don't need to perfectly clean every spelling mistake. The model is robust by design and generalizes better

# FASTTEXT: ALTERNATIVE WORD EMBEDDINGS

## — Training FastText with Skip-Gram with Negative Sampling

— Example: *"I need to **debug** code"*

### — Input representation

- For the target word ("debug"), we calculate its vector by summing the vectors of its n-grams and the whole word.
- N-Grams (n=3): <de, deb, ebu, bug, ug>, <debug>
- The n-gram bug already carries semantic meaning related to the target!

### — Context representation

- For the context word ("code"), we take its whole word vector directly from the embedding table
- We do not use n-grams for the output context words

### — Negative sampling

- For the actual pair ("debug", "code"), we sample 5 random negative words (noise).

### — Scoring

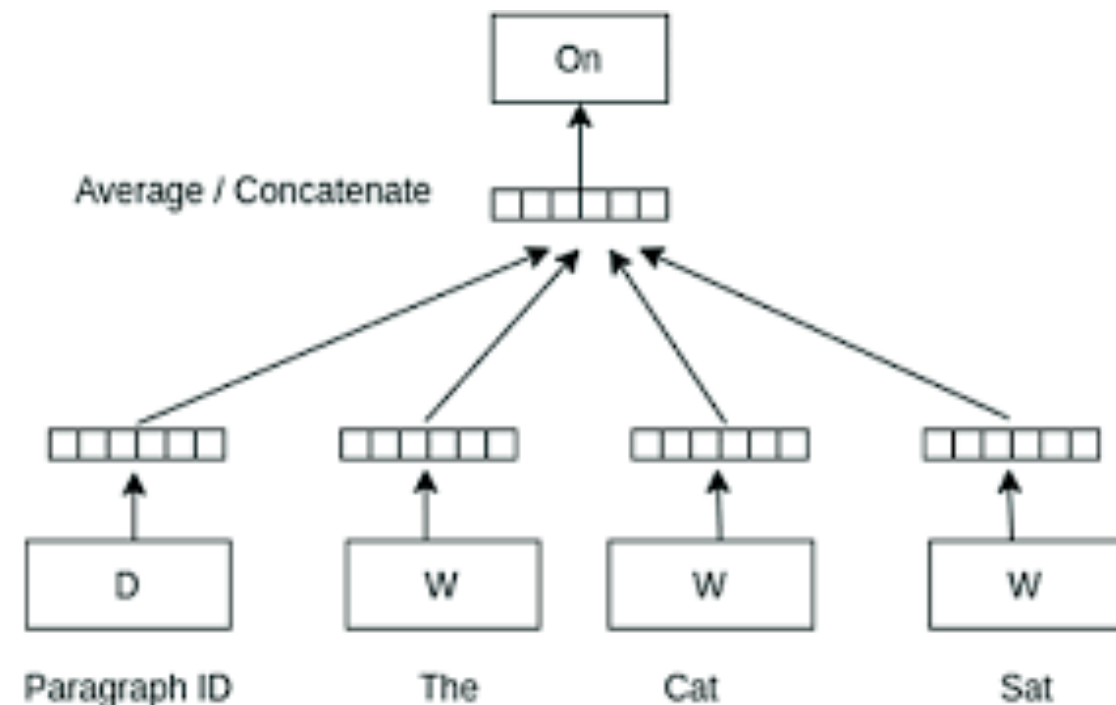
- We calculate the dot product between the input and the context/negative Vectors, then apply a Sigmoid function.

### — Update with stochastic gradient descent

- Maximize score for "debug" and "code". Minimize score for "debug" and "piano".
- The optimizer pulls the n-grams (like bug) closer to "code" and pushes them away from "piano".

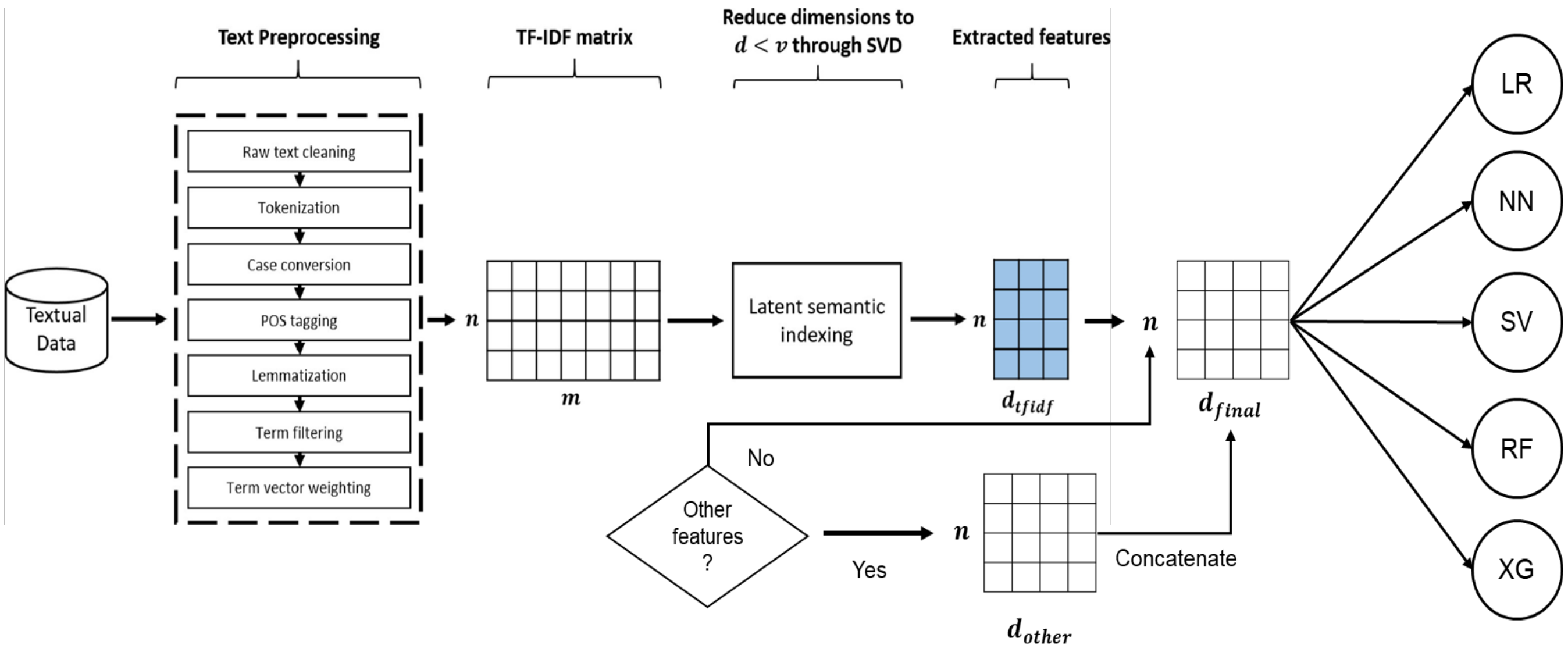
# PARAGRAPH VECTOR-DISTRIBUTED MEMORY (PV-DM) MODEL

- The PV-DM model (often referred to as **Doc2Vec**) extends the standard Word2Vec model by adding a **paragraph vector** that acts as a memory unit (Le & Mikolov, 2014)
- This vector acts as a "missing memory" that remembers the topic of the paragraph.
- The input:
  - Context words: Sampled from a sliding window
  - Paragraph vector: Shared across all windows in the same document
- The output:
  - The model predicts the target word based on **both** the local words and the global paragraph vector
- Inference:
  - We cannot simply "calculate" the vector for a new, unseen document
  - We create a **random vector** for the new document, **freeze** the trained word weights, and run **gradient descent** to learn the new paragraph vector that minimizes error.



# MACHINE LEARNING MODELING

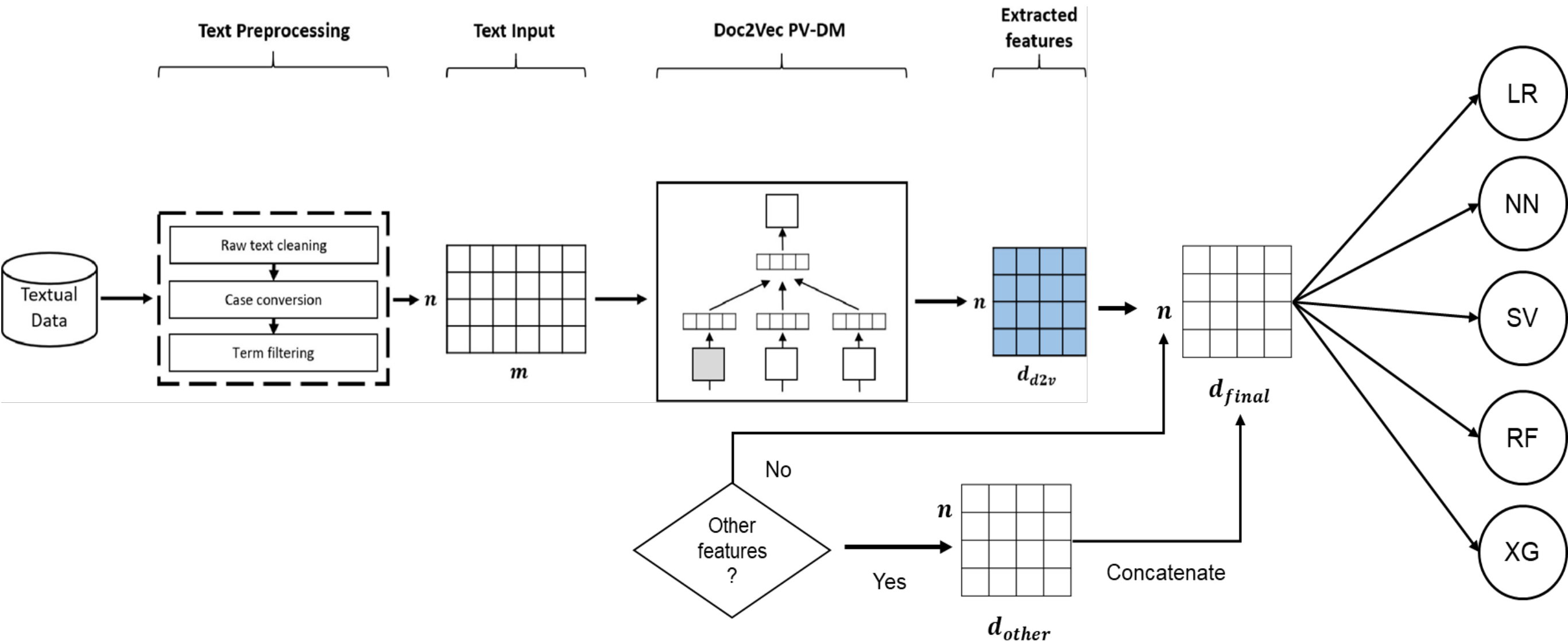
## — Traditional machine learning models set-up with SVD





# MACHINE LEARNING MODELING

## – Traditional machine learning models set-up with embeddings





# MODEL EVALUATION

## — Evaluation

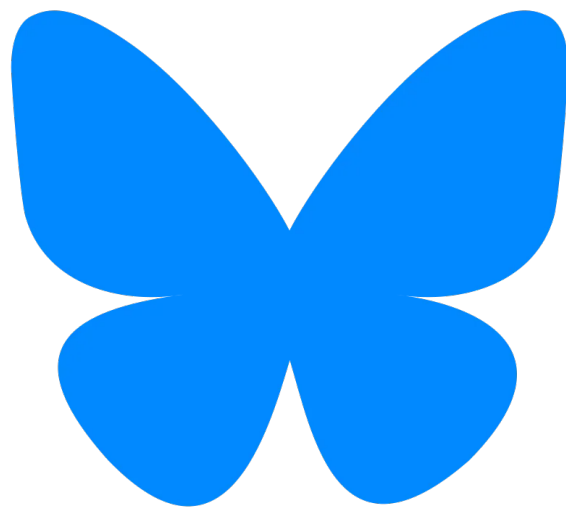
- Evaluate performance of our predictions vs actual sentiment on the test set
- Differs for binary vs multiclass solutions
- Based on the confusion matrix
  - Cut-off dependent
    - Accuracy (sensitivity and specificity)
    - Precision and recall
    - F1 measure, G-mean
  - Cut-off independent
    - AUC
- Traditional ML models achieve an accuracy of 80-87% for **binary sentiment** classification, transformer-based models can achieve an accuracy of 95% on benchmark data sets
- In **multi-class settings**, traditional ML models plateau at 65-75% and transformers can reach up to 75-85% (depending on the number of classes)

# REMEMBER

Post about this lecture:

[#SMWA2026](#)

[#Lecture5](#)



# SOCIAL MEDIA AND WEB ANALYTICS

Prof Dr Matthias Bogaert